SIMATS ENGINEERING

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA12	**Course Title:**	Computer Architecture
CO Assessed:	CO3 – Precision (BL3)	**Assessment:**	Experiments
Weightage:	45%	**Total Marks:**	100
CO3 Statement:	Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT).		
Student Name:	V.Sriram	**Reg. No.:**	192524037
Section / Batch:	2025	**Date:**	10-08-2026

Instructions to Students

- Identify the relevant concepts of register transfers, ALU operations, bus organization, pipelining, hazards, and superscalar processing.
- Analyze the execution of data transfers, arithmetic and logic operations, instruction processing, and parallel execution.
- Apply suitable techniques such as multiple buses, pipelining, stalling, forwarding, branch prediction, and speculative execution.
- Compute performance measures such as execution time, clock cycles, speedup, throughput, and resource utilization.
- Interpret the results by evaluating performance improvements, bottlenecks, and the suitability of each architectural technique.

QUESTIONS

1. Analyze a processor datapath that performs register transfers and logical operations such as XOR, NOT, and comparison, and examine the control signals and execution sequence required for each operation.

2. Develop a bus-based system for transferring data between the processor, memory, and I/O devices, and analyze how bus width, transfer rate, and bus contention influence system performance.

3. Simulate a pipelined processor with arithmetic, memory, and branch instructions, and evaluate the effects of pipeline depth on execution time, throughput, and overall speedup.

4. Construct a pipeline containing instruction dependencies and branch conditions, and analyze the performance impact of data and control hazards using stalling, forwarding, and branch prediction techniques.

5. Develop a superscalar processor model that supports multiple execution units and out-of-order instruction processing, and evaluate how instruction scheduling and resource allocation improve instruction throughput.

Code of Conduct

I V.Sriram(192524037) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: V.Sriram

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Q. No	Understanding of Concepts(25)	Experimental Design (30)	Use of Tools(20)	Analysis of Results (15)	Documentation (10)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	**/ 30**	**/ 20**	**/ 15**	**/ 10**	**/ 100**

Course Coordinator **Faculty Incharge**

1)

Aim

To analyze the execution of register transfer, XOR, NOT, and comparison operations in a processor datapath and identify the required control signals.

1. Processor Datapath

A basic processor datapath consists of:

- General-purpose registers
- Multiplexer (MUX)
- ALU
- Control Unit
- Internal bus
- Status/flag registers

2. Register Transfer Operation

Consider:

R1 ← R2

This means the contents of register **R2** are transferred to **R1**.

Execution sequence

1. Select R2 as the source register.
2. Place R2 data on the internal bus.
3. Enable R1 for loading.
4. On the clock edge, R1 receives the data.

Control signals

Control Signal	Function
R2out	Places R2 on bus
R1in	Loads bus data into R1
Clock	Synchronizes transfer

3. XOR Operation

Consider:

R3 ← R1 XOR R2

Example

R1 = 10101100

R2 = 11001010

Perform XOR:

```
  10101100
⊕11001010
  ----------
  01100110
```

Therefore:

R3 = 01100110

Execution sequence

1. Select R1 as ALU input A.
2. Select R2 as ALU input B.
3. Select XOR operation in ALU.
4. Execute XOR.

5. Store result in R3.

Control signals

Signal	Function
R1out	Sends R1 to ALU
R2out	Sends R2 to ALU
ALU_XOR	Selects XOR operation
R3in	Loads result into R3
Clock	Synchronizes operation

4. NOT Operation

Consider:

R2 ← NOT R1

Example:

R1 = 10110010

NOT operation:

NOT 10110010

↓

01001101

Therefore:

R2 = 01001101

Execution sequence

1. Read R1.
2. Send R1 to ALU.
3. Select NOT operation.
4. Complement every bit.
5. Store result in R2.

Control signals

Signal	Function
R1out	Sends R1 to ALU
ALU_NOT	Selects NOT operation
R2in	Loads result into R2
Clock	Synchronizes operation

5. Comparison Operation

Consider:

CMP R1, R2

The ALU performs:

R1 - R2

but normally does not store the subtraction result. Instead, it updates the **status flags**.

Example:

R1 = 20

R2 = 15

Then:

20 - 15 = 5

Since the result is positive:

Zero Flag = 0

Sign Flag = 0

Carry/Borrow = 0

Therefore:

R1 > R2

Control signals

Signal	Function
R1out	First operand
R2out	Second operand
ALU_SUB	Performs comparison
FlagWrite	Updates flags
Clock	Synchronizes operation

Overall Control Signal Table

Operation	Source	ALU Operation	Destination
R1 ← R2	R2	Transfer	R1
R3 ← R1 XOR R2	R1, R2	XOR	R3
R2 ← NOT R1	R1	NOT	R2
CMP R1,R2	R1, R2	SUBTRACT	Flags

Result

The processor performs register transfers and logical operations by coordinating the **register file, bus, ALU, multiplexers, control signals, and status flags**. Each operation requires a specific sequence of source selection, ALU control, destination loading, and clock synchronization.

2)

Aim

To design a bus-based processor system and analyze
the effect of bus width, transfer rate, and bus contention on performance.

1. Bus-Based System

A computer system uses three major buses:

1. Address Bus
2. Data Bus
3. Control Bus

2. Address Bus

The address bus carries the address of the required memory or I/O location.
For example, a **32-bit address bus** can address:
2^{32} locations
which is:
4 GB address space

3. Data Bus

The data bus transfers actual data.
For example, a:
32-bit data bus
can transfer:
32 bits = 4 bytes
per bus transfer.

4. Control Bus

The control bus carries control signals such as:

- Read

- Write
- Memory Enable
- I/O Enable
- Interrupt
- Clock

5. Effect of Bus Width

A wider bus transfers more data in one cycle.

Suppose:

Data = 1024 bytes

Bus width = 32 bits = 4 bytes

Number of transfers:

1024 / 4 = 256 transfers

If the bus is 64 bits:

64 bits = 8 bytes

Number of transfers:

1024 / 8 = 128 transfers

Therefore, increasing bus width reduces the number of transfers.

6. Effect of Transfer Rate

Assume:

Bus width = 32 bits

Clock frequency = 100 MHz

Data transferred per cycle:

32 bits = 4 bytes

Theoretical bandwidth:

4 bytes × 100,000,000

= 400,000,000 bytes/s

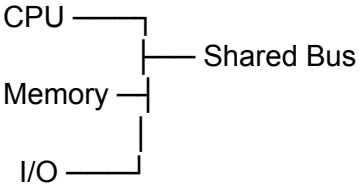
Therefore:

Bandwidth = 400 MB/s

7. Bus Contention

Bus contention occurs when multiple devices attempt to use the same bus at the same time.

Example:



If CPU and I/O both request the bus simultaneously, only one should be allowed to transmit.

Bus arbitration

A bus arbiter can decide which device gets access.

Possible techniques:

- Priority arbitration
- Round-robin arbitration
- Daisy-chain arbitration

8. Performance Analysis

Parameter	Effect
Wider bus	More data per transfer
Higher transfer rate	Higher bandwidth

Bus contention	Reduces effective bandwidth
Arbitration delay	Adds waiting time
Narrow bus	More transfer cycles

3)

Aim

To simulate a pipelined processor containing arithmetic, memory, and branch instructions and analyze pipeline depth, execution time, throughput, and speedup.
The assignment specifically requires evaluating execution time, throughput, and overall speedup.

1. Five-Stage Pipeline

Use a standard 5-stage pipeline:

Stage	Meaning
IF	Instruction Fetch
ID	Instruction Decode
EX	Execute
MEM	Memory Access
WB	Write Back

2. Example Instructions

Consider:
I1: ADD R1,R2,R3
I2: LOAD R4,0(R5)
I3: SUB R6,R7,R8
I4: STORE R6,0(R9)
I5: BEQ R1,R4,LABEL
These represent:

- Arithmetic instruction
- Memory instruction
- Arithmetic instruction
- Memory instruction
- Branch instruction

3. Pipeline Execution

Cycle	I1	I2	I3	I4	I5
1	IF				
2	ID	IF			
3	EX	ID	IF		
4	MEM	EX	ID	IF	
5	WB	MEM	EX	ID	IF
6		WB	MEM	EX	ID
7			WB	MEM	EX
8				WB	MEM
9					WB

For 5 instructions and 5 pipeline stages:

Total cycles = k + n - 1
where:
k = 5
n = 5
Therefore:
Cycles = 5 + 5 - 1
= 9 cycles

4. Non-Pipelined Execution
If each instruction requires 5 cycles:
5 instructions × 5 cycles
= 25 cycles
Therefore:
Non-pipelined = 25 cycles
Pipelined = 9 cycles

5. Speedup
Formula:
Speedup = Non-pipelined time / Pipelined time
Therefore:
Speedup = 25 / 9

Speedup ≈ 2.78
So the pipeline gives approximately:
2.78× speedup
for this example.

6. Effect of Pipeline Depth
Consider 20 instructions.
5-stage pipeline
Cycles = 5 + 20 - 1
= 24 cycles
10-stage pipeline
Cycles = 10 + 20 - 1
= 29 cycles
The deeper pipeline has more initial pipeline overhead.
However, after the pipeline becomes full, both can potentially complete approximately one instruction per cycle.

7. Throughput
Throughput is the number of instructions completed per unit time.
For an ideal pipeline:
Throughput ≈ 1 instruction/cycle
After the pipeline is filled.
If clock frequency is:
2 GHz
then ideal throughput is approximately:
2 billion instructions/second
assuming one instruction completes every cycle.

8. Analysis of Pipeline Depth

Shallow pipeline
Advantages:

- Lower branch penalty
- Simpler control
- Less pipeline overhead

Disadvantages:

- Lower maximum clock frequency

Deep pipeline
Advantages:

- Can support higher clock frequency
- More work is divided into smaller stages

Disadvantages:

- Greater branch penalty
- More pipeline registers
- More hazard complexity
- More misprediction cost

Result
Pipelining improves instruction throughput by overlapping instruction execution. Increasing pipeline depth can increase clock frequency, but excessive depth increases branch penalties, control complexity, and pipeline overhead.

4)

Aim
To analyze **data hazards and control hazards** in a pipeline and evaluate the effectiveness of **stalling, forwarding, and branch prediction**.
The assignment explicitly asks for instruction dependencies, branch conditions, and performance analysis using these three techniques.

1. Instruction Sequence
Consider:
I1: ADD R1,R2,R3
I2: SUB R4,R1,R5
I3: AND R6,R4,R7
I4: BEQ R6,R0,LABEL
I5: OR R8,R9,R10

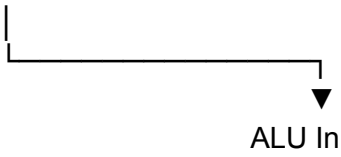
2. Data Dependencies
I1 → I2
I1 produces R1
I2 uses R1
Therefore:
I1 → I2
I2 → I3
I2 produces R4
I3 uses R4
Therefore:
I2 → I3
I3 → I4
I3 produces R6
I4 uses R6

Therefore:
I3 → I4
These create **Read After Write (RAW)** dependencies.

3. Hazard Without Forwarding
Consider:
I1: ADD R1,R2,R3
I2: SUB R4,R1,R5
I2 requires R1 before I1 has written it back.
Pipeline:

Cycle	I1	I2
1	IF	
2	ID	IF
3	EX	ID
4	MEM	STALL
5	WB	STALL
6		EX
7		MEM
8		WB

Two stall cycles are required.

4. Forwarding
Forwarding sends the result directly from a later pipeline stage to an earlier stage without waiting for register write-back.


With forwarding, the dependency can often be resolved without the two-cycle stall.
Example:
I1: ADD R1,R2,R3
I2: SUB R4,R1,R5
The result of I1 can be forwarded directly to I2's ALU input.

5. Stalling
If forwarding cannot resolve the dependency, the processor inserts a **stall**.
A stall is also called a:
Pipeline bubble
Example:
Instruction → Instruction → Bubble → Instruction
Stalling is simple but reduces throughput.

6. Control Hazard
Consider:
I4: BEQ R6,R0,LABEL
I5: OR R8,R9,R10
The processor does not immediately know whether the branch is taken.
This creates a **control hazard**.

7. Branch Prediction
The processor predicts whether the branch is:
Taken
or
Not Taken
Correct prediction
The pipeline continues normally.
Incorrect prediction
The incorrectly fetched instructions are discarded.
This is called:
Pipeline flush

8. Performance Comparison
Suppose a branch has a penalty of:
3 cycles
and there are 100 branches.
If prediction is always correct:
Penalty = 0 cycles
If 20 branches are mispredicted:
Penalty = 20 × 3
= 60 cycles
Therefore, reducing branch mispredictions directly improves performance.

9. Comparison

Technique	Advantage	Disadvantage
Stalling	Simple and safe	Reduces throughput
Forwarding	Reduces data stalls	Requires extra hardware
Branch prediction	Reduces control stalls	Wrong prediction causes flush
Perfect prediction	Maximum performance	Not practically achievable

10. Overall Analysis
Without forwarding
More stalls occur.
With forwarding
Data hazards are reduced.
With branch prediction
Control hazards are reduced.
Therefore, combining:
Forwarding + Stalling + Branch Prediction
provides much better pipeline performance than using stalling alone.

Result
Data hazards caused by instruction dependencies can significantly reduce pipeline performance. **Forwarding** minimizes unnecessary stalls, while **branch prediction** reduces control-hazard penalties. Stalling remains necessary when a dependency cannot be resolved by forwarding.

5)
Aim

To develop a simple **superscalar processor model** with multiple execution units and **out-of-order execution**, and analyze its effect on instruction throughput.
The assignment asks specifically for multiple execution units, out-of-order processing, instruction scheduling, resource allocation, and throughput improvement.

1. Superscalar Processor
A superscalar processor can execute **more than one instruction per clock cycle**.
2. Multiple Execution Units
Assume the processor contains:
ALU 1 → Integer arithmetic
ALU 2 → Integer/logic operations
Load/Store → Memory operations
Branch Unit → Branch instructions
Therefore, different instructions can execute simultaneously.

3. Example Instruction Sequence
Consider:
I1: ADD R1,R2,R3
I2: MUL R4,R5,R6
I3: LOAD R7,0(R8)
I4: SUB R9,R10,R11
Assume:

- ADD = 1 cycle
- MUL = 3 cycles
- LOAD = 2 cycles
- SUB = 1 cycle

4. In-Order Execution
If instructions are executed strictly in order:
I1 → I2 → I3 → I4
Approximate execution:
1 + 3 + 2 + 1
= 7 execution cycles

5. Out-of-Order Execution
The processor checks which instructions are ready.
For example:
Cycle 1:
I1 → ALU1
I2 → ALU2

Cycle 2:
I3 → Load/Store

Cycle 3:
I4 → ALU1
Multiple independent instructions can execute concurrently.

6. Instruction Scheduling
The scheduler examines:

- Operand availability

- Execution-unit availability
- Instruction dependencies
- Priority
- Resource conflicts

For example:

Instruction Required Unit

ADD ALU1

MUL ALU2

LOAD Load Unit

SUB ALU1

The scheduler assigns instructions to available units.

7. Resource Allocation

Suppose:

ALU1 = busy

ALU2 = free

Load Unit = free

A multiplication instruction can be sent to ALU2 while another instruction is using ALU1.

This improves hardware utilization.

8. Throughput Calculation

Suppose a processor executes:

100 instructions

Single-issue processor

If it completes approximately one instruction per cycle:

100 cycles

2-wide superscalar processor

Ideal execution:

100 / 2 = 50 cycles

Thus ideal speedup:

100 / 50 = 2×

Actual speedup will be lower because of:

- Data dependencies
- Branches
- Resource conflicts
- Cache misses
- Pipeline stalls

9. Comparison

Feature	**Scalar**	**Superscalar**
Instructions/cycle	~1	>1
Execution units	One/few	Multiple
Scheduling	Simple	Advanced
Parallel execution	Limited	High
Throughput	Lower	Higher
Hardware complexity	Lower	Higher

10. Advantages of Out-of-Order Execution

1. Better resource utilization
Idle execution units can execute independent instructions.
2. Higher throughput
Multiple instructions can execute simultaneously.
3. Hides latency
A long operation does not necessarily stop all other independent instructions.
4. Better performance
The processor can exploit **Instruction-Level Parallelism (ILP)**.

11. Limitations
Out-of-order superscalar processors require additional hardware such as:

- Instruction scheduler
- Reservation stations
- Register renaming
- Reorder buffer
- Multiple execution units
- Dependency checking logic

Therefore, performance improves at the cost of **hardware complexity, power consumption, and design complexity**.

Result
A superscalar processor with multiple execution units and out-of-order execution can execute independent instructions concurrently. Effective instruction scheduling and resource allocation increase instruction throughput and improve processor performance. However, dependencies, branch instructions, and resource conflicts limit the ideal speedup.